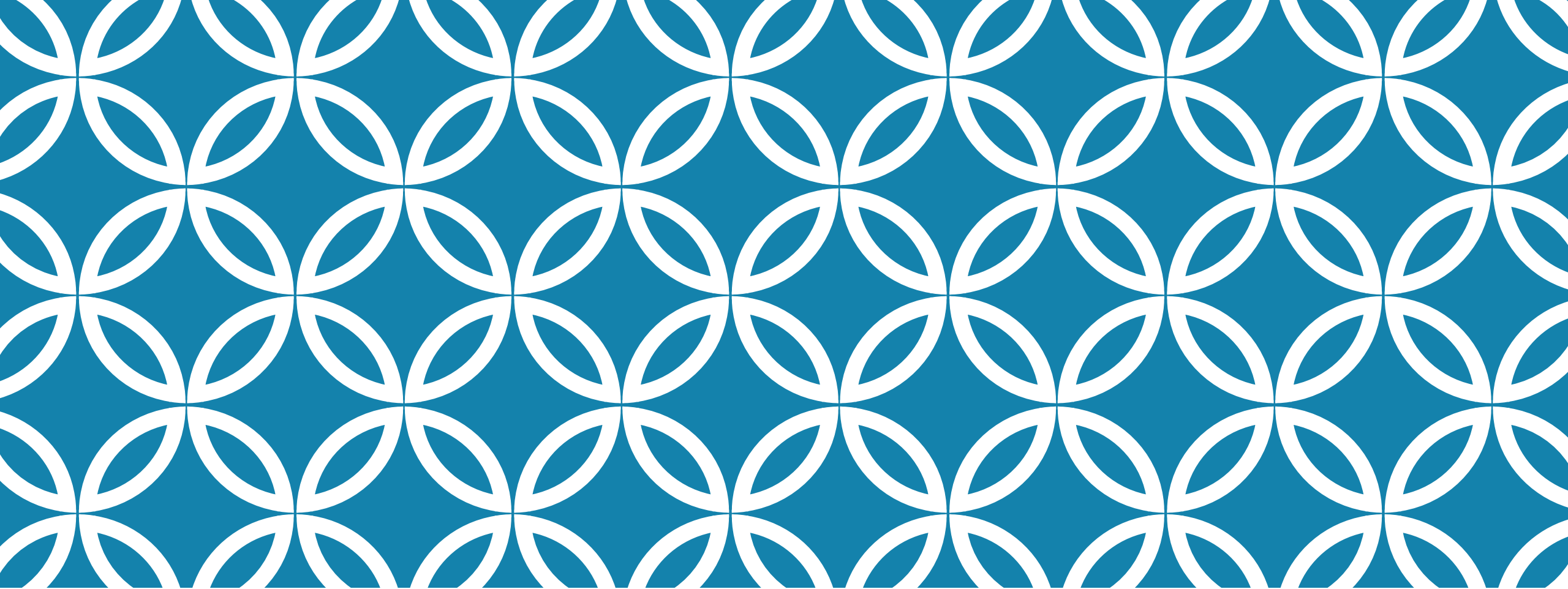




I8086/8088 PROGRAMAÇÃO

Arquitetura de Computadores

ESTRUTURA DE UM PROGRAMA ASM

.model <memória>

.stack <tamanho>

.data

<dados>

.code

<código>

end <ponto inicial>

ESTRUTURA: MODELO DE MEMÓRIA

A diretiva `.model` define o modelo de memória que será utilizado pelo programa

- **TINY**: um segmento de 64KB para tudo
- **SMALL**: um segmento de código e outro de dados
- **COMPACT**: um segmento de código e múltiplos de dados
- **MEDIUM**: múltiplos segmentos de código e um de dados
- **LARGE**: múltiplos segmentos de código e dados
- **HUGE**: igual ao **LARGE**, exceto que permite criar matrizes maiores que 64KB
- **FLAT**: um único segmento de 4GB (somente 32 bits)

ESTRUTURA: TAMANHO DO STACK

A diretiva `.stack` reserva um espaço de memória para o segmento de pilha.

- Desnecessário quando o modelo de memória for **tiny**
- Default é 1KB

Cuidado ao definir esta diretiva pois diversas chamadas de sub-rotinas mais o empilhamento de registradores (salvamento de contexto) pode requerer uma grande área da memória

ESTRUTURA: SEGMENTO DE DADOS

A diretiva **.data** determina o início do segmento de dados onde todas as definições de dados da memória estão armazenadas

As variáveis (dados) definem-se através de diretivas indicando apenas quantos bytes ocupam os seus valores sem especificar o respectivo tipo

- DB : define 1 byte (número ou caractere)
- DW: define 2 bytes
- DD: define 4 bytes
- DQ: define 8 bytes
- DT: define 10 bytes

ESTRUTURA: SEGMENTO DE DADOS

Exemplos

```
Alfa      DB  ?           ; não inicializada
Beta      DW  ?           ; não inicializada
Epsilon   DW  5           ; constante 05H
Iota      DB  'HELLO'     ; contém 48 45 4C 4C 4F H
```

Outras diretivas

- DUP: repete um determinado valor indicado
- EQU: define uma constante

```
Mu        DB 100 DUP 0    ; aloca e inicializa 100 bytes com 0 (zero)
VETOR     DB 48h, 65h, 00h ; vetor de 3 posições
CONT      EQU 25          ; constante CONT = 25
```

ESTRUTURA: SEGMENTO DE CÓDIGO

A diretiva **.code** determina o início do segmento de código

Deve ter um ponto de início do programa (este ponto é um rótulo)

```
.code  
    inicio:  
        <código>  
end inicio
```

ESTRUTURA: SEGMENTO DE CÓDIGO

No Turbo Assembler, é necessário inicializar o segmento de dados

```
.code  
    inicio:  
        mov AX, @DATA  
        mov DS, AX  
        ...  
end inicio
```


ESTRUTURA: SEGMENTO DE CÓDIGO

Linha de comando

ROTULO: OPERAÇÃO OPERANDO(S) ; COMENTÁRIO

- Exemplo

INÍCIO: mov AX, @DATA ; inicializa o reg AX

O campo `ROTULO` pode ser um rótulo de instrução, um nome de sub-rotina ou um nome de variável (31 caracteres). Deve iniciar com uma letra e conter somente letras, números e os caracteres `? . @ _ : $ %`

ESTRUTURA: SEGMENTO DE CÓDIGO

Para finalizar um programa necessita-se chamar a interrupção do DOS que realiza esta operação

```
.code
    inicio:
        mov AX, @DATA
        mov DS, AX
        ...
        mov al, 00;    código de retorno
        mov ah, 4CH
        int 21H
end inicio
```

ESTRUTURA: SEGMENTO DE CÓDIGO

Diretivas do TASM

▪ **OFFSET**

- Retorna o deslocamento de um nome (dado ou rotina) dentro do seu segmento
- Por exemplo,

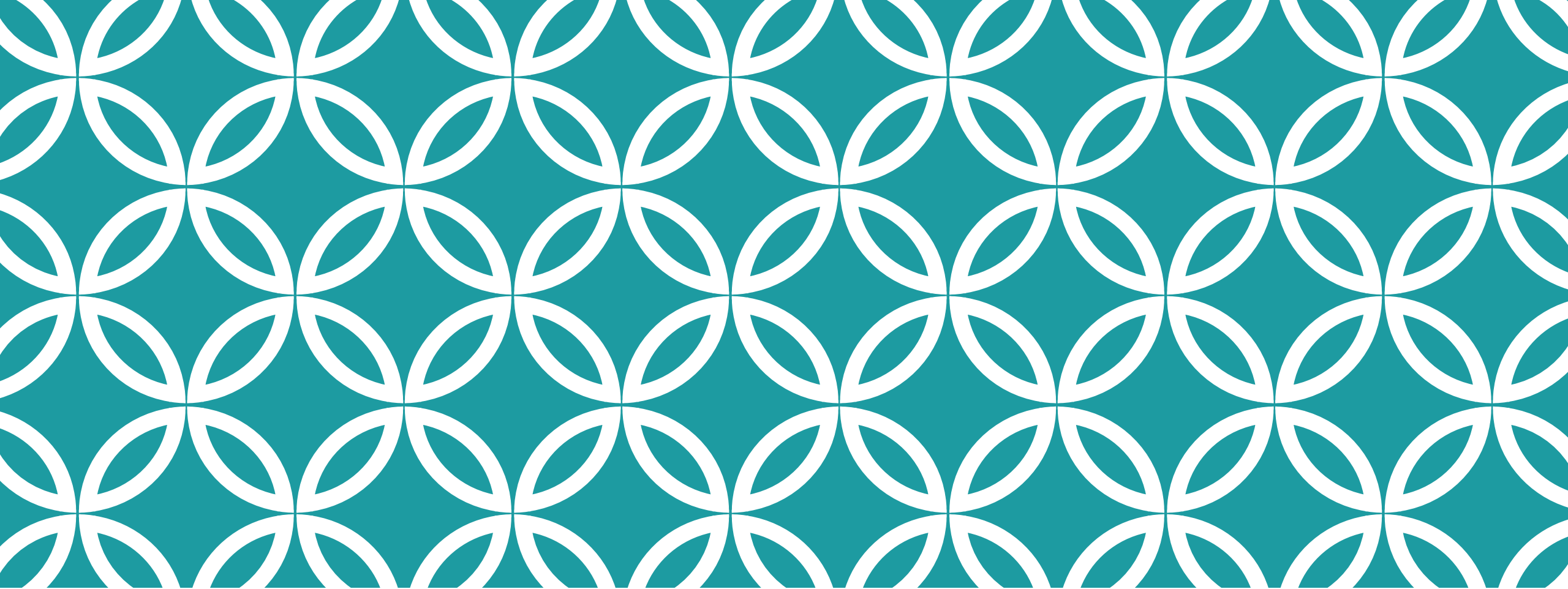
mov bx, offset NOME

Retorna o endereço de memória apontado por NOME

▪ **@CODE**

- Retorna o endereço do segmento de código
- Por exemplo,

mov ax, @CODE



CONJUNTO DE INSTRUÇÕES DO I8086

Arquitetura de Computadores

CONJUNTO DE INSTRUÇÕES DO I8086

As instruções do 8086 podem ser classificadas em 6 grupos

- Transferência de dados
- Aritméticas
- Lógicas e de Deslocamento
- Manipulação de strings
- Desvio
- Controle do processador

CONJUNTO DE INSTRUÇÕES DO I8086

Instruções de Transferência de Dados

MOV

XCHG

PUSH/PUSHF

POP/POPF

XLAT

IN

OUT

LEA

LES

LDS

LAHF

SAHF

INSTRUÇÃO MOV

Mnemônico	Significado	Formato	Operação	Flags afetados
MOV	Mover	Mov D, F	(F) → (D)	Nenhum

Destino	Fonte
Memória	Acumulador
Acumulador	Memória
Registrador	Registrador
Registrador	Memória
Memória	Registrador
Registrador	Imediato
Memória	Imediato
Reg Segmento	Registrador 16
Reg Segmento	Memória16
Registrador 16	Reg Segmento
Memória	Reg Segmento

NÃO PODE

Memória	→	Memória
Imediato	→	Reg Segmento
Reg Segmento	→	Reg Segmento
*	→	CS

EX: **MOV AL, BL**
MOV CX, BX

INSTRUÇÕES DE TRANSFERÊNCIA DE DADOS

Mnemônico	Significado	Formato	Operação	Flags afetados
XCHG	Trocar	XCHG D, F	$D \Leftrightarrow F$	Nenhum

Destino	Fonte
Acumulador	Registrador 16
Registrador	Registrador
Registrador	Memória
Memória	Registrador

NÃO PODE

Registradores de Segmento
Memória com Memória

EX:

XCHG [1234h], BX ; Troca conteúdos do end. 1234h e BX

XCHG AX, BX ; Troca os conteúdos de AX e BX

XCHG BL, VARIABEL ; Troca o conteúdo de BL com o
; conteúdo do offset 'VARIABEL'

XCHG AL, BL ; Troca o conteúdo de AL com BL

INSTRUÇÕES DE TRANSFERÊNCIA DE DADOS

Mnemônico	Significado	Formato	Operação	Flags afetados
LEA	Carrega Endereço Efetivo (EA)	LEA Reg16, EA	EA → (Reg16)	Nenhum
LDS	Carrega registrador e DS	LDS Reg16, MEM32	(MEM32) → (Reg16) (Mem32+2) → (DS)	
LES	Carrega registrador e ES	LES Reg16, MEM32	(MEM32) → (Reg16) (Mem32+2) → (DS)	

LEA SI, DATA ou MOV SI, OFFSET DATA

INSTRUÇÕES DE TRANSFERÊNCIA DE DADOS

Exemplos para LDS e LEA

```
DATA DW 1000H
```

```
DATAY DW 5000H
```

...

```
LEA SI, DATA
```

```
MOV DI, OFFSET DATAY;
```

```
LEA BX, [DI];
```

```
MOV BX, DI;
```

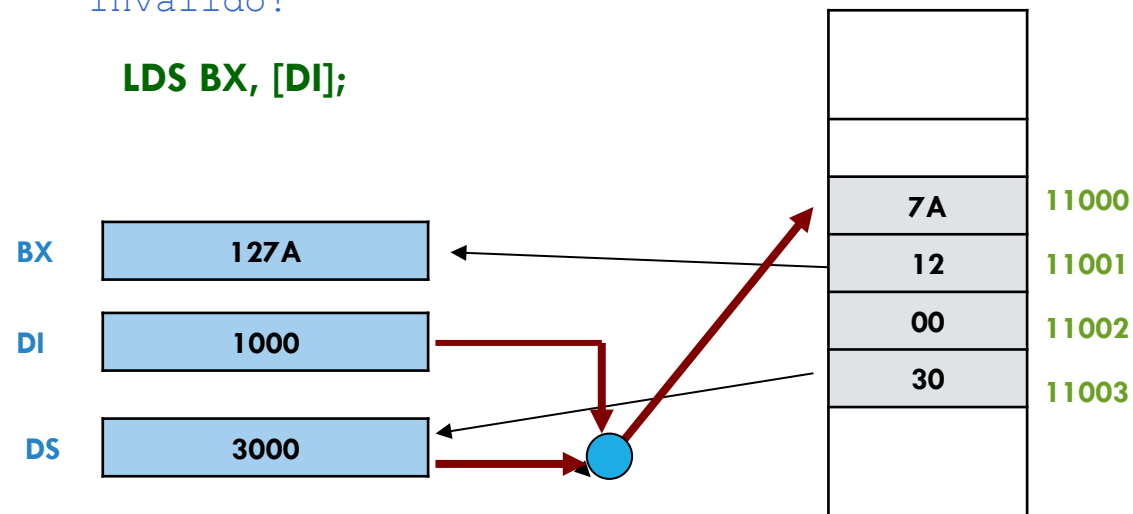
```
LEA BX, DI;
```

Isto é mais eficiente que o anterior.

É o mesmo que isto mas leva menos ciclos.

Inválido!

LDS BX, [DI];



LES é similar ao LDS exceto que ele carrega o ES

INSTRUÇÕES DE TRANSFERÊNCIA DE DADOS

Mnemônico	Significado	Formato	Operação	Flags afetados
XLAT	Traduz	XLAT	[AL+BX] → AL	Nenhum

EX: BX = 0100H AL = 0DH
XLAT ⇔ [010DH] → AL

```
Tabela DB 30h,31h,32h,33h,34h,35h,36h,37h,38h,39h
        DB 41h,42h,43h,44h,45h,46h
```

```
...
MOV AL,0Ch      ; converter 0Ch para caractere ASCII 'C'
LEA BX,TABELA   ; BX recebe o offset de TABELA
XLAT            ; é feita a conversão e
...            ; AL recebe 43h = 'C'
```

INSTRUÇÕES DE TRANSFERÊNCIA DE DADOS

Mnemônico	Significado	Formato	Operação	Flags afetados
PUSH	Coloca palavra na pilha	PUSH F	$SP \leftarrow SP - 2$ $[SP] \leftarrow F$	Nenhum
POP	Retira palavra da pilha	POP D	$D \leftarrow [SP]$ $SP \leftarrow SP + 2$	Nenhum
PUSHF	Coloca flags na pilha	PUSHF	$SP \leftarrow SP - 2$ $[SP] \leftarrow \text{Flags}$	Nenhum
POPF	Retira flags da pilha	POPF	$\text{Flags} \leftarrow [SP]$ $SP \leftarrow S + 2$	Todos

Operando (F or D)
Registrador Registrador de Segmento (CS é ilegal) Memória

INSTRUÇÕES DE TRANSFERÊNCIA DE DADOS

Mnemônico	Significado	Formato	Operação	Flags afetados
LAHF	Carrega AH com os flags SF, ZF, AF, PF, e CF	LAHF	$(AH_{7,6,4,2,0}) \leftarrow (SF, ZF, AF, PF, CF)$	Nenhum
SAHF	Coloca o conteúdo de AH nos flags SF, ZF, AF, PF, e CF	POP D	$(SF, ZF, AF, PF, CF) \leftarrow (AH_{7,6,4,2,0})$	SF, ZF, AF, PF, CF

INSTRUÇÕES DE TRANSFERÊNCIA DE DADOS

Mnemônico	Significado	Formato	Operação	Flags afetados
IN	Transfere dados de uma porta especificada para o acumulador	IN Acumulador, Porta	Acumulador $\leftarrow$ Porta	Nenhum
OUT	Transfere dados do acumulador para uma porta	OUT Porta, Acumulador	Porta $\leftarrow$ Acumulador	Nenhum

Acumulador	Porta
AL	Imediato 8
AX	Imediato 8
AL	DX
AX	DX

EX:

```
MOV DX, 72H; Do canal 72H
IN AL, DX ; Recebe um valor("byte")
...
MOV DX, 378H ; Endereço de porta
OUT DX, AX ; Enviar dados aos
              ; canais 378 e 379
```

CONJUNTO DE INSTRUÇÕES DO I8086

Instruções Aritméticas

ADD

SUB

INC

MUL

DIV

CBW

NEG

ADC

SBB

DEC

IMUL

IDIV

CWD

CMP

INSTRUÇÕES ARITMÉTICAS

Mnemônico	Significado	Formato	Operação	Flags afetados
ADD	Adição	ADD D, F	$D = D + F$ $CF = carry$	CF, OF, PF, SF, ZF
ADC	Adição com Carry	ADC D, F	$D = D + F + CF$ $CF = carry$	CF, OF, PF, SF, ZF
INC	Incremento por um	INC D	$D = D + 1$	OF, PF, SF, ZF

Destino	Fonte
Registrador	Registrador
Registrador	Memória
Memória	Registrador
Registrador	Imediato
Memória	Imediato

INSTRUÇÕES ARITMÉTICAS

Ex.1

```
ADD AX, 2  
ADC AX, 2
```

Ex.2

```
INC BX  
INC WORD PTR [BX]
```

Ex.3

```
ASCII 0-9 = 30-39h  
MOV AX, 38H; (código ASCII para nro 8)  
ADD AL, 39H; (código ASCII para nro 9) AL=71h
```

INSTRUÇÕES ARITMÉTICAS

Mnemônico	Significado	Formato	Operação	Flags afetados
SUB	Subtração	SUB D, F	$(D)-(F) \rightarrow (D)$ Borrow $\rightarrow (CF)$	CF, OF, PF, SF, ZF
SBB	Subtração com borrow	SBB D, F	$(D)-(F)-(CF) \rightarrow (D)$	CF, OF, PF, SF, ZF
DEC	Decrementar por um	DEC D	$(D)-1 \rightarrow (D)$	OF, PF, SF, ZF
NEG	Negar (C-2)	NEG D		CF, OF, PF, SF, ZF

Destino	Fonte
Registrador	Registrador
Registrador	Memória
Memória	Registrador
Registrador	Imediato
Memória	Imediato

INSTRUÇÕES ARITMÉTICAS

Ex.1

MOV BL, 28H

MOV AL, 83H

SUB AL, BL; AL=5BH

Ex.2

MOV AX, 38H

SUB AL, 39H; AX=00FF CF=1

Ex.3

BX=003AH

NEG BX; (BX)=00000000000111010

Comp 2 = 1111111111000110 = FFC6H

CF=1

INSTRUÇÕES ARITMÉTICAS

Mnemônico	Significado	Formato	Operação	Flags afetados
MUL	Multiplicação SEM sinal	MUL F	Se F tiver 8 bits (resultado deve ter 16 bits) $AX = AL * F$	Se as metades mais significativas do produto (AH ou DX) contiverem dígitos significativos, CF e OF serão ligados
IMUL	Multiplicação COM sinal	IMUL F	Se F tiver 16 bits (resultado deve ter 32 bits) $DX:AX = AX * F$	

```
MOV BL, 35H
```

```
MOV AL, 85H
```

```
XOR AH, AH
```

```
MUL BL
```

```
; AX = 85H * 35H = 1B89H
```

```
MOV BL, 35H
```

```
MOV AL, 85H
```

```
XOR AH, AH
```

```
IMUL BL
```

```
; AX = 85H * 35H (produto com sinal)
```

```
;      = E689H
```

Fonte

Registrador

Memória

INSTRUÇÕES ARITMÉTICAS

Mnemônico	Significado	Formato	Operação	Flags afetados
DIV	Divisão SEM sinal	DIV F	Se F tiver 8 bits $AL = AX / F$ (quociente) $AH = AX \% F$ (resto) Se F tiver 16 bits $AX = DXAX / F$ (quociente) $DX = DXAX \% F$ (resto)	Se as metades mais significativas do produto (AH ou DX) contiverem dígitos significativos, CF e OF serão ligados
IDIV	Multiplicação COM sinal	IDIV F		

A instrução de divisão não afeta flag algum

Fonte

Registrador

Memória

Se o quociente não couber em AL ou AX (conforme o tamanho da divisão), o sistema acusará overflow na divisão

INSTRUÇÕES ARITMÉTICAS

```
MOV AX, 85H ; 13310
```

```
MOV BL, 35H ; 5310
```

```
DIV BL
```

```
; AH = 1BH ; 2710
```

```
; AL = 02H
```

No caso de uma atribuição ao registrador AL, garanta que AH esteja zerado

```
XOR AH, AH
```

```
MOV AL, 85H
```

```
XOR DX, DX
```

```
MOV AX, 2385H ; 909310
```

```
MOV BX, 35H ; 5310
```

```
DIV BX
```

```
; AX = ABH ; 2710
```

```
; DX = 1EH ; 3010
```

Se não estiver usando DX na divisão, garanta que ele esteja zerado

INSTRUÇÕES ARITMÉTICAS

A instrução de divisão a seguir vai causar erro por overflow, pois o resultado B6DH é maior que FFH (sendo impossível ser armazenado no registrador AL)

```
MOV AX, 0F000H ; 6144010
```

```
MOV BX, 15H ; 2110
```

```
DIV BL
```

INSTRUÇÕES ARITMÉTICAS

No caso de divisão com sinal, deve-se garantir que AX ou DX:AX representem corretamente o valor negativo (ou positivo)

Antes da divisão, deve-se estender o bit de sinal de valores em AL utilizando a instrução CBW e em AX utilizando a instrução CWD

Mnemônico	Significado	Formato	Operação	Flags afetados
CBW	Converte byte para word	CBW	Se bit $AL_7 = 1$ então AH = 0FFH Senão AH = 0H	Nenhum
CWD	Converte word para double word	CWD	Se bit $AX_{15} = 1$ então DX = 0FFFFH Senão DX = 0H	Nenhum

INSTRUÇÕES ARITMÉTICAS

Quando o dividendo é armazenado no registrador AL, deve-se sempre utilizar a instrução CBW antes da instrução IDIV

```
MOV AL, 85H    ; -12310
CBW            ; AX = FF85H
MOV BL, 35H    ; 5310
IDIV BL
; AH = FEH    ; -2710
; AL = FEH    ; -210
```

No caso de uma divisão de 8 bits, mas o registrador AX é setado com o dividendo, não é necessária a utilização da instrução CBW

```
MOV AX, -123   ; FF85H
MOV BL, 35H    ; 5310
IDIV BL
; AH = FEH    ; -2710
; AL = FEH    ; -210
```

INSTRUÇÕES ARITMÉTICAS

Quando o dividendo é armazenado no registrador AX, deve-se sempre utilizar a instrução CWD antes da instrução IDIV

- Como CWD preenche DX com o bit de sinal de AX, não é necessário inicializar o registrador DX

```
MOV AX, -1250 ; F816H
```

```
CWD           ; DX:AX = FFFFFFF816H
```

```
MOV BL, 7     ; 5310
```

```
IDIV BX
```

```
; AX = FF4EH ; -17810
```

```
; DX = FFFCH ; -410
```

INSTRUÇÕES ARITMÉTICAS

Mnemônico	Significado	Formato	Operação	Flags afetados
CMP	Comparar	CMP D, F	D-F é utilizado para setar ou resetar os flags	CF, AF, OF, PF, SF, ZF

Destino	Fonte
Registrador	Registrador
Registrador	Memória
Memória	Registrador
Registrador	Imediato
Memória	Imediato
Acumulador	Imediato

EX:

MOV AL, 7
CMP AL, 5

MOV AL, 99H; -103
CMP AL, 1BH; +27

CONJUNTO DE INSTRUÇÕES DO I8086

Instruções de Desvios

- Desvios Incondicionais

JMP

CALL

RET

- Desvios Condicionais

JE / JZ

JNE / JNZ

JB / JNAE

JBE / JNA

JNB / JAE

JNBE / JA

JP / JPE

JNP / JPO

JL / JNGE

JLE / JNG

JNL / JGE

JNLE / JG

JO

JNO

JS

JNS

JCXZ

- Desvios por Interrupção

INT

IRET

INSTRUÇÕES DE DESVIOS

Mnemônico	Significado	Formato	Operação	Flags afetados
JMP	Salto Incondicional	JMP Operando	O salto é feito para o endereço especificado pelo operando	Nenhum

Operando
Short Rótulo
Near Rótulo
Far Rótulo
Ptr Memória 16
Ptr Registrador 16
Ptr Memória 32

Existem 3 tipos de saltos (desvio no fluxo do código)

- NEAR
- SHORT
- FAR

INSTRUÇÕES DE DESVIOS

NEAR: salto para uma instrução dentro do segmento de código atual

- **Direto (3 bytes):** o operando é um endereço (rótulo ou constante) da instrução alvo que será codificado em um deslocamento de 16 bits (-32768 a +32767) relativo ao valor atual do registrador IP (próxima instrução)

```
JMP ROTULO
```

```
JMP 234H
```

- **Indireto (2 bytes):** o operando é um registrador de 16 bits ou um endereço de memória de 16 bits que especifica um deslocamento absoluto no segmento de código. Este deslocamento é carregado no registrador IP

```
JMP AX      ; IP = AX
```

```
JMP [BX]    ; IP = DS:BX
```

INSTRUÇÕES DE DESVIOS

SHORT: salto do tipo NEAR (direto), mas limitado a um deslocamento relativo de 8 bits com sinal (instrução ocupa somente 2 bytes)

- O operando é um endereço (rótulo ou constante) da instrução alvo do desvio que será codificado em um deslocamento de 8 bits (-128 a +127) relativo ao valor atual do registrador IP (próxima instrução)

FAR: salto para uma instrução localizada em outro segmento de código

- **Direto (6 bytes):** o operando é um endereço (rótulo ou endereço lógico) da instrução alvo (IP = bytes 2 e 3, CS = bytes 4 e 5 do formato da instrução)

```
JMP ROTULO  
JMP 1234H:3456h
```

- **Indireto (2 bytes):** o operando é um ponteiro de memória que possui e 32 bits que especifica um endereço lógico que deve ser carregado nos registradores CS e IP

```
JMP DWORD PTR [DI]      ; IP = WORD PTR DS:DI  
                        ; CS = WORD PTR DS:(DI +2)
```

INSTRUÇÕES DE DESVIOS

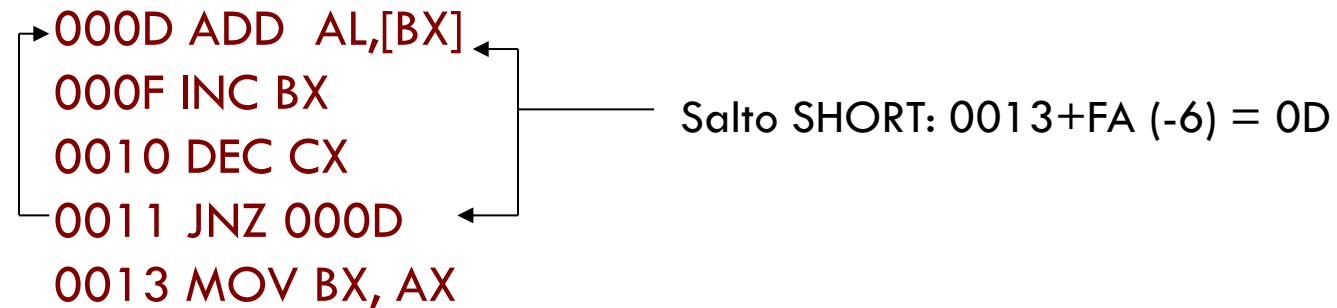
Os saltos condicionais desviam o fluxo do programa com base no estado de determinados flags

Geralmente, são utilizados após uma instrução de comparação (CMP ou TEST) ou após instruções lógicas ou aritméticas (alteram os flags)

INSTRUÇÕES DE DESVIOS

Como a instrução possui um tamanho de 2 bytes, a instrução alvo do salto é especificado com um deslocamento relativo de 8 bits com sinal

- O deslocamento relativo é somado ao IP
- A instrução alvo pode estar em um deslocamento de 127 bytes a frente do valor corrente do IP ou 128 bytes antes do valor corrente do IP



INSTRUÇÕES DE DESVIOS

Mnemônico	Operação	Instrução se condição contrária
JS	Salta se $SF = 1$ (RESULTADO NEGATIVO)	JNS
JO	Salta se $OF = 1$ (OVERFLOW)	JNO
JZ	Salta se $ZF = 1$ (RESULTADO IGUAL A ZERO)	JNZ
JE		JNE
JC	Salta se $CF = 1$ (RESULTADO IGUAL A ZERO)	JNC
JP	Salta se $PF = 1$ (PARIDADE PAR)	JNP
JPE	Salta se $PF = 1$ (PARIDADE PAR)	JPO
JCXZ	Salta se $CX = 0$	

INSTRUÇÕES DE DESVIOS

As seguintes instruções são utilizadas com base na comparação de entre dados **SEM** sinal (após cmp OP1, OP2)

Mnemônico	Equivalente	Operação	Flags	Instrução se condição contrária
JA	JNBE	Salta se acima – $op1 > op2$	CF=0 e ZF=0	JNA
JAE	JNB, JNC	Salta se acima ou igual – $op1 \geq op2$	CF=0	JNAE
JB	JNAE, JC	Salta se abaixo – $op1 < op2$	CF=1	JNB
JBE	JNA	Salta se abaixo ou igual – $op1 \leq op2$	CF=1 ou ZF=1	JNBE

INSTRUÇÕES DE DESVIOS

As seguintes instruções são utilizadas com base na comparação de entre dados **COM** sinal (após cmp OP1, OP2)

Mnemônico	Equivalente	Operação	Flags	Instrução se condição contrária
JG	JNLE	Salta se maior que – $op1 > op2$	SF=OF e ZF=0	JNG, JLE
JGE	JNL	Salta se maior que ou igual – $op1 \geq op2$	SF=OF	JNGE, JL
JL	JNGE	Salta se menor que – $op1 < op2$	SF<>OF	JNL, JGE
JLE	JNG	Salta se maior que ou igual – $op1 \leq op2$	ZF=1 ou SF<>OF	JNLE, JG

INSTRUÇÕES DE DESVIOS: EXEMPLO

Um fragmento da rotina que recebe um valor em AL e retorna a soma dos valores de 1 até AL em BX. Todos os outros registradores devem ser inalterados

```
Início:  PUSH AX      ; SP = SP - 2 e coloca AX na pilha
          XOR AH, AH   ; zerar AH
          XOR BX, BX   ; BX = 0
For:     ADD BX, AX    ; BX = BX + AX
          DEC AL       ; AL ← AL - 1
          JNZ FOR      ; salta para FOR, se ZF=0
          POP AX       ; recupera AX da pilha, SP = SP + 2
          RET          ; retorno de sub-rotina
```

INSTRUÇÕES DE DESVIOS

Mnemônico	Significado	Formato	Operação	Flags afetados
CALL	Chamada de subrotina	CALL operando	Execução continua a partir do endereço da subrotina especificada pelo operando. As informações necessárias (CS e IP) para voltar ao programa que fez a chamada são empilhados	Nenhum
RET	Retornar de uma subrotina	RET RET n	Recupera IP (e CS) da pilha. $SP \leftarrow SP + 2$ No caso de RET n, $SP \leftarrow SP + n$	Nenhum

Operando

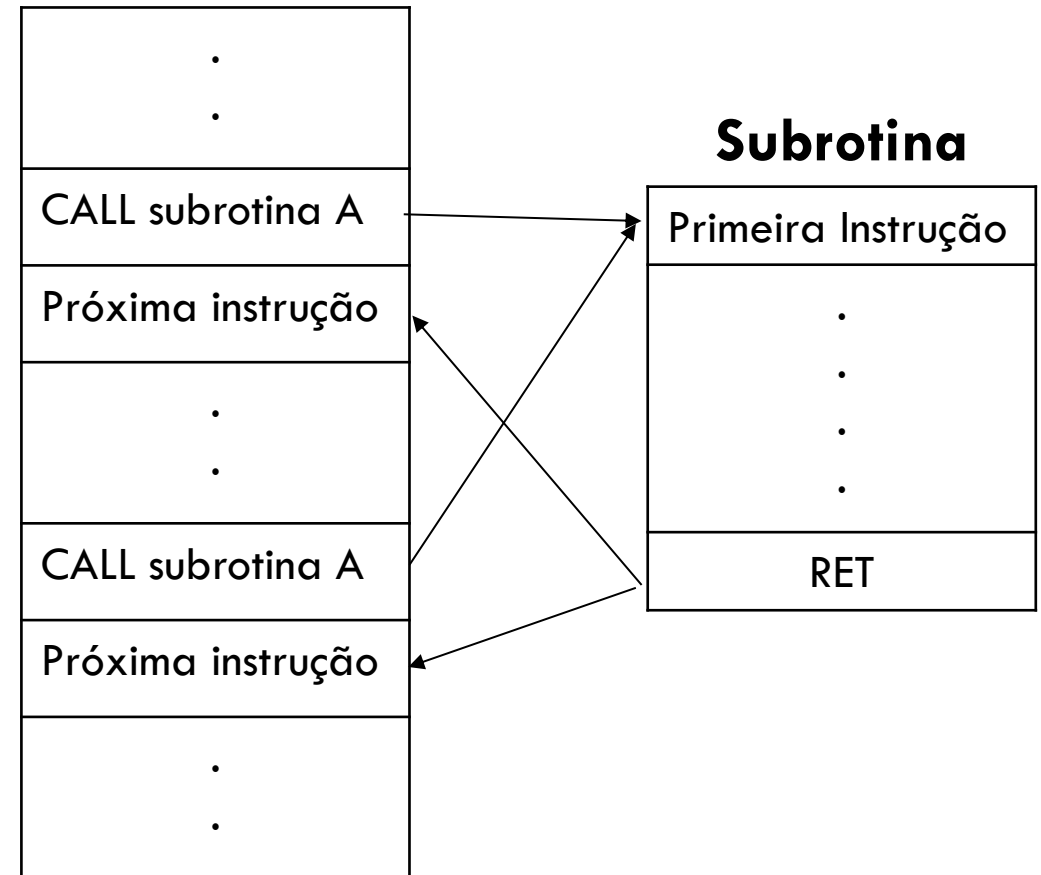
Near-proc
Far-proc
Memptr16
Regptr16
Memptr32

INSTRUÇÕES DE DESVIOS

Ao executar a instrução CALL, o endereço da instrução que a segue é empilhado

- Este endereço empilhado é chamado de endereço de retorno

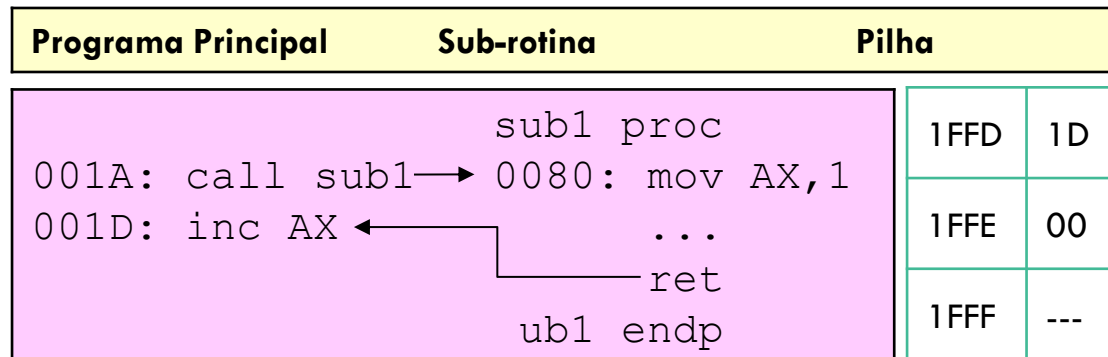
Na subrotina, ao executar a instrução RET, o endereço que está na pilha é copiado para o registrador IP (e assim o programa continua a partir da instrução que segue a chamada da subrotina)



INSTRUÇÕES DE DESVIOS

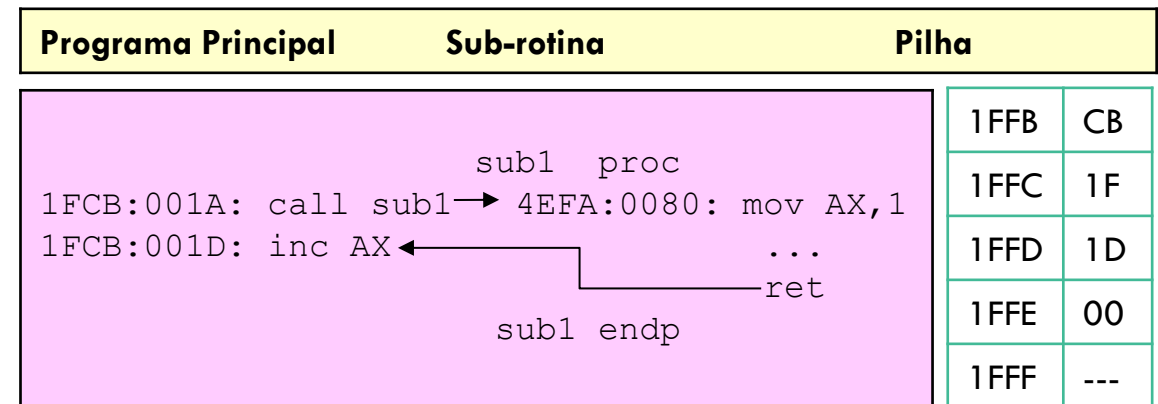
Na execução de uma sub-rotina NEAR (ou SHORT), i.e., dentro do mesmo segmento de código

- Empilha somente o registrador IP
- Carrega o deslocamento da sub-rotina no registrador IP



Na execução de uma sub-rotina FAR, i.e., instrução CALL e sub-rotina estão em segmentos diferentes

- Empilha os registradores IP e CS
- Carrega o deslocamento da subrotina no registrador IP e o CS da sub-rotina



CONJUNTO DE INSTRUÇÕES DO I8086

Instruções de Repetição

LOOP

LOOPZ/LOOPE

LOOPNZ/LOOPNE

INSTRUÇÕES DE REPETIÇÃO

Mnemônico	Significado	Formato	Operação
LOOP	Repita	LOOP rótulo	$(CX) \leftarrow (CX) - 1$ Salto é realizado para rótulo se $(CX) \neq 0$; caso contrário, executa a próxima instrução.
LOOPE	Repita enquanto igual	LOOPE rótulo	$(CX) \leftarrow (CX) - 1$ Salto é realizado para rótulo se $(CX) \neq 0$ e $ZF = 1$; caso contrário, executa a próxima instrução.
LOOPZ	Repita enquanto zero	LOOPZ rótulo	$(CX) \leftarrow (CX) - 1$ Salto é realizado para rótulo se $(CX) \neq 0$ e $ZF = 0$; caso contrário, executa a próxima instrução.
LOOPNE	Repita enquanto não igual	LOOPNE rótulo	$(CX) \leftarrow (CX) - 1$ Salto é realizado para rótulo se $(CX) \neq 0$ e $ZF = 1$; caso contrário, executa a próxima instrução.
LOOPNZ	Repita enquanto não zero	LOOPNZ rótulo	$(CX) \leftarrow (CX) - 1$ Salto é realizado para rótulo se $(CX) \neq 0$ e $ZF = 0$; caso contrário, executa a próxima instrução.

Operando
SHORT Rótulo

INSTRUÇÕES DE REPETIÇÃO: EXEMPLO

Um fragmento da rotina que recebe um valor em AL e retorna a soma dos valores de 1 até AL em BX. Todos os outros registradores devem ser inalterados

```
Início: PUSH CX      ; SP = SP - 2 e coloca <CX> na pilha
        XOR CH, CH    ; zerar CH
        XOR BX, BX    ; BX = 0
        MOV CL, AL    ; CL = AL
For:    ADD BX, CX     ; BX = BX + CX
        LOOP For      ; CX = CX - 1 e salta para For, se CX ≠ 0
        POP CX        ; recupera CX da pilha, SP = SP + 2
        RET           ; retorno de sub-rotina
```

INSTRUÇÕES DE REPETIÇÃO: EXEMPLO

Buscar o caractere espaço em branco (ASCII = 20H) numa string de tamanho DL com endereço inicial em STR

```
        MOV CX, DL          ; inicialização da variável de controle
        MOV SI, -1          ; inicialização do índice
        MOV AL, 20H         ; AL = ' '
NEXT:   INC SI               ; SI = SI + 1
        CMP AL, [SI+STR]    ; AL - [SI + STR], flags são modificados
        LOOPNZ NEXT         ; CX = CX - 1 e salta para NEXT
                                ; se CX ≠ 0 e ZF = 0
        JNZ NOT_FOUND       ; salta para NOT_FOUND se espaço
                                ; em branco não encontrado
```

INSTRUÇÕES LÓGICAS

Lógicas

AND

OR

NOT

XOR

TEST

Deslocamento

SHL

SAL

SHR

SAR

Rotação

ROR

ROL

RCL

RCR

INSTRUÇÕES LÓGICAS

Mnemônico	Significado	Formato	Operação	Flags afetados
AND	E lógico	AND D,F	$D = D \cdot F$	OF, SF, ZF, PF, CF AF indefinido
OR	OU lógico	OR D,F	$D = D + F$	
XOR	OU Exclusivo	XOR D,F	$D = D \oplus F$	
TEST	E lógico	TEST D,F	$D \cdot F$	
NOT	NÃO	NOT D	$D = \bar{D}$	Nenhum

Destino	Fonte	Destino (NOT)
Registrador	Registrador	Registrador
Registrador	Memória	Memória
Memória	Registrador	
Registrador	Imediato	
Memória	Imediato	
Acumulador	Imediato	

INSTRUÇÕES LÓGICAS

AND

- Utiliza qualquer modo de endereçamento exceto memória-memória e registradores de segmento
- Utilizado para resetar certos bits (máscara)

$xxxx\ xxxx\ AND\ 0000\ 1111 = 0000\ xxxx$
(set os primeiro quatro bits)

- Exemplos: `AND BL, 0FH`
 `AND AL, [345H]`

OR

- Utilizado para setar certos bits

$xxxx\ xxxx\ OR\ 0000\ 1111 = xxxx\ 1111$
(Seta os 4 bits mais significativos)

INSTRUÇÕES LÓGICAS

XOR

- Utilizado para inverter bits

$$\text{xxxx xxxx XOR 0000 1111} = \text{xxxxx'x'x'x'}$$

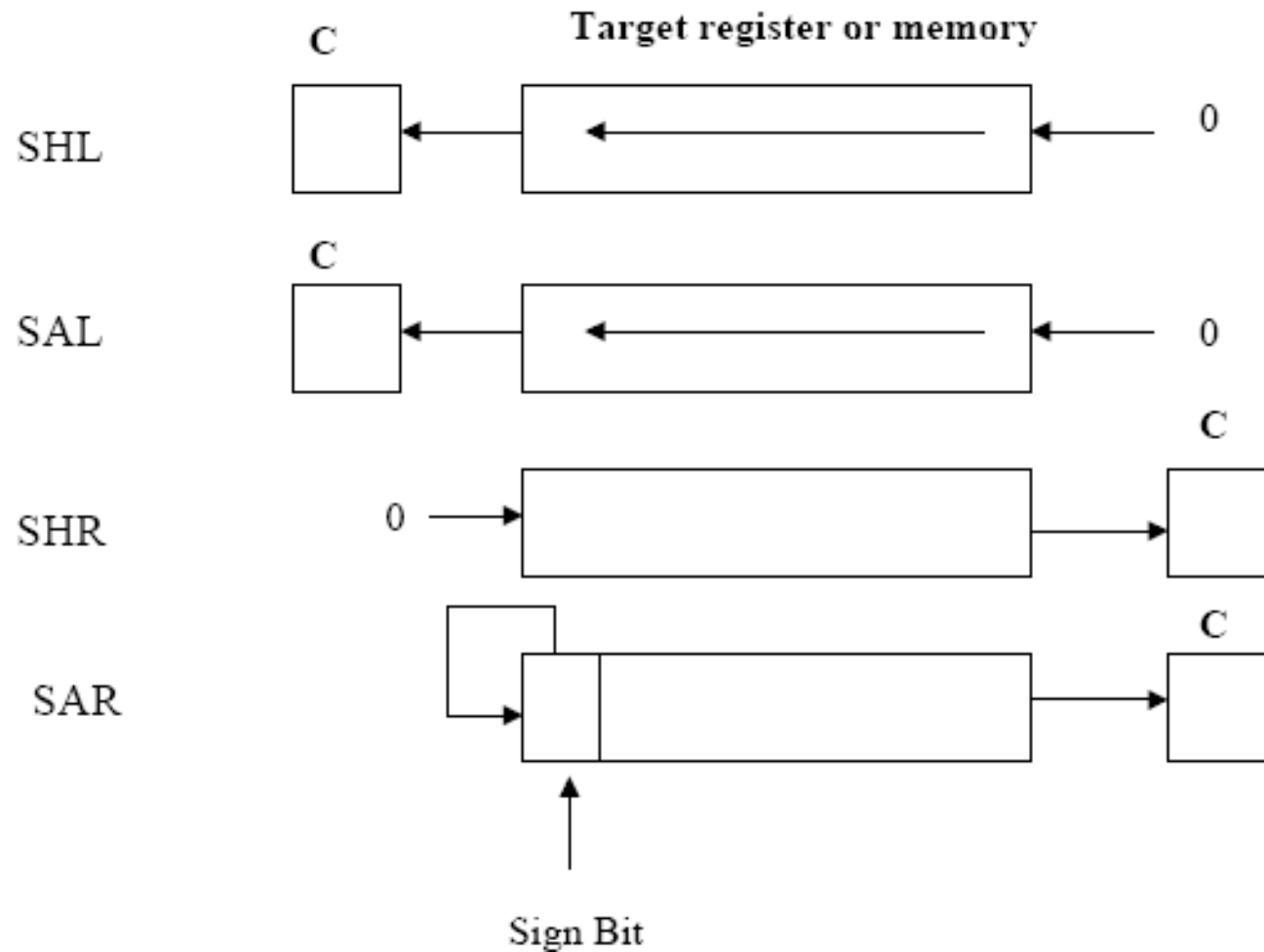
- Exemplo: ressetar bits 0 e 1, setar bits 6 e 7, e inverter bit 5 do registrador CL:

```
AND CL, 0FCH;    1111 1100B
OR CL, 0C0H;      1100 0000B
XOR CL, 020H;      0010 0000B
```


INSTRUÇÕES LÓGICAS

Mnemônico	Significado	Formato	Operação	Flags afetados
SAL/SHL	Deslocamento aritmético / lógico para a esquerda	SAL/SHL D, Cont	Desloca o (D) para a esquerda Cont bits e preenche as posições à direita de bits com zeros	CF,PF,SF,ZF AF indefinido OF indefinido (Cont ≠ 1)
SHR	Deslocamento lógico para a direita	SHR D, Cont	Desloca o (D) para a direita Cont bits e preenche as posições à esquerda de bits com zeros	CF,PF,SF,ZF AF indefinido OF indefinido (Cont ≠ 1)
SAR	Deslocamento aritmético para a direita	SAR D, Cont	Desloca o (D) para a direita Cont bits e preenche as posições à esquerda de bits com o bit mais significativo (sinal)	CF,PF,SF,ZF AF indefinido OF indefinido (Cont ≠ 1)

INSTRUÇÕES LÓGICAS



Destino	Cont
Registrador	1
Registrador	CL
Memória	1
Memória	CL

INSTRUÇÕES LÓGICAS

Como multiplicar AX por 10 utilizando deslocamentos?

```
SHL AX, 1
```

```
MOV BX, AX
```

```
MOV CL, 2
```

```
SHL AX, CL
```

```
ADD AX, BX
```

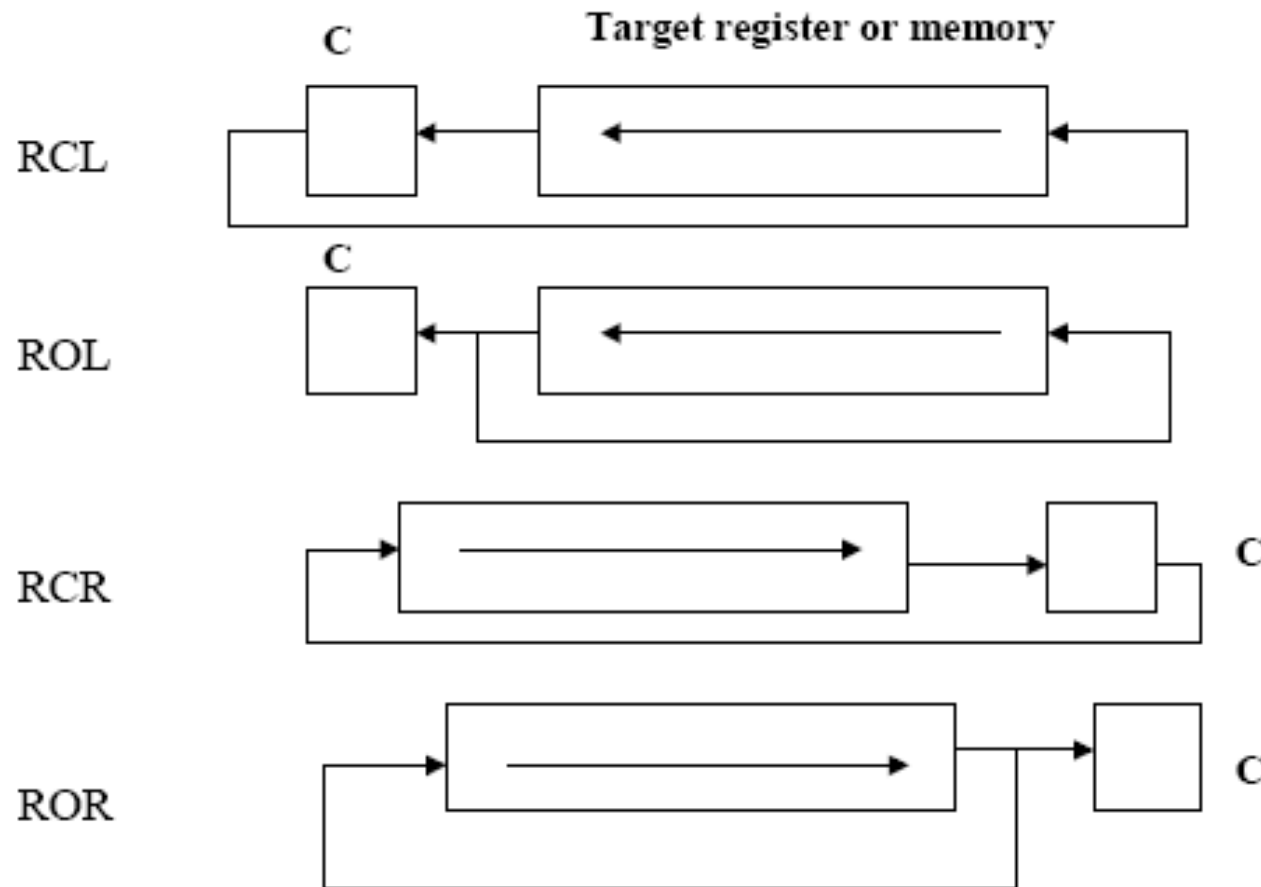
Qual é o resultado de SAR CL, 1 se CL inicialmente contém B6H? **DBH**

Qual é o resultado de SHL AL, CL se AL contém 75H e CL contém 3? **A8H**

INSTRUÇÕES LÓGICAS

Mnemônico	Significado	Formato	Operação	Flags afetados
ROL	Rotação para a esquerda	ROL D, Cont	Rotaciona (D) para a esquerda pelo número de bits igual a Cont. Cada bit deslocado para fora volta na posição à direita.	CF OF indefinido (Cont $\neq$ 1)
ROR	Rotação para a direita	ROR D, Cont	Rotaciona (D) para a direita pelo número de bits igual a Cont. Cada bit deslocado para fora volta na posição à esquerda.	CF OF indefinido (Cont $\neq$ 1)
RCL	Rotação para a esquerda com carry	RCL D, Cont	Igual a ROL, exceto que o CF é utilizado com (D) para a rotação	CF OF indefinido (Cont $\neq$ 1)
RCR	Rotação para a direita com carry	RCR D, Cont	Igual a ROR, exceto que o CF é utilizado com (D) para a rotação	CF OF indefinido (Cont $\neq$ 1)

INSTRUÇÕES LÓGICAS



Destino	Cont
Registrador	1
Registrador	CL
Memória	1
Memória	CL

INSTRUÇÕES LÓGICAS

Escrever um programa que conta o número de 1 em um byte e escreve-o em BL

```
DATA1 DB 97          ; 61h
SUB BL, BL           ; limpa BL - contador de 1s
MOV CX, 8            ; Contador
MOV AL, DATA1

AGAIN: ROL AL,1       ; rotaciona à esquerda uma vez
      JNC NEXT        ; verifica por 1
      INC BL          ; se CF=1 então incrementa contador

NEXT:  LOOP AGAIN      ; se não acabou, volta
      NOP
```

INSTRUÇÕES DE MANIPULAÇÃO DE STRINGS

MOVSB

MOVSW

STOSB

STOSW

LODSB

LODSW

CMPSB

CMPSW

SCASB

SCASW

Repetição

REP

REPNE

REPNZ

REPE

REPZ

INSTRUÇÕES DE MANIPULAÇÃO DE STRINGS

Mnemônico	Significado	Formato	Operação	Flags afetados
MOVS	Move string	MOVSB MOVSW	$[ES:DI] \leftarrow [DS:SI]$ $SI \leftarrow SI \pm 1 \text{ ou } 2$ $DI \leftarrow DI \pm 1 \text{ ou } 2$	Nenhum
CMPS	Compara string	CMPSB CMPSW	Seta os flags conforme $[DS:SI] - [ES:DI]$ $SI \leftarrow SI \pm 1 \text{ ou } 2$ $DI \leftarrow DI \pm 1 \text{ ou } 2$	CF,PF,AF,ZF,SF,OF
SCAS	Compara string com um elemento	SCASB SCASW	Seta os flags conforme $(AL \text{ ou } AX) - [ES:DI]$ $DI \leftarrow DI \pm 1 \text{ ou } 2$	CF,PF,AF,ZF,SF,OF
LODS	Copia string	LODSB LODSW	$(AL \text{ ou } AX) \leftarrow [DS:SI]$ $SI \leftarrow SI \pm 1 \text{ ou } 2$	Nenhum
STOS	Armazena string	STOSB STOSW	$[ES:DI] \leftarrow (AL \text{ ou } AX)$ $DI \leftarrow DI \pm 1 \text{ ou } 2$	Nenhum

DF = 0 → Incrementar DF = 1 → Decrementar

INSTRUÇÕES DE MANIPULAÇÃO DE STRINGS

Programa que copia STR1 para STR2

```
        MOV AX, DS           ; Cópia DS para
        MOV ES, AX          ; ES
        MOV SI, OFFSET STR1  ; endereço da STR1
        MOV DI, OFFSET STR2  ; endereço da STR2
        MOV CX, 128          ; tamanho da STR1
        CLD                  ; DF = 0
MOVE:    MOVSB
        LOOP MOVE
```

INSTRUÇÕES DE MANIPULAÇÃO DE STRINGS

Programa a seguir converte para minúscula uma string (finalizada com 0) com endereço em DX

```

                MOV     SI, DX      ; Inicializa índices
                MOV     DI, DX      ; fonte e destino
                MOV     AX, DS      ; Cópia DS (segmento fonte)
                MOV     ES, AX      ; para ES (segmento destino)
                CLD
NextCh:          LODSB              ; Carrega prox caractere em AL
                CMP     AL, 41h     ;
                JB      NotUC       ; Salta se abaixo de 'A'
                CMP     AL, 5Bh     ;
                JA      NotUC       ; ou acima de 'Z'
                ADD     AL, 20h      ; Converte MAIUSC para minus
NotUC:          STOSB              ; Armazena caractere modificado
                CMP     AL, 0       ; Checa fim de string
                JNE     NextCh      ; Faz proximo caracter se não for
```

INSTRUÇÕES DE MANIPULAÇÃO DE STRINGS

Prefixo	Usado com	Significado
REP	MOVS STOS	Repita enquanto não for fim da string ($CX \neq 0$)
REPE/REPZ	CMPS SCAS	Repita enquanto não for fim de string e as strings são iguais ($CX \neq 0$ e $ZF=1$)
REPNE/REPNZ	CMPS SCAS	Repita enquanto não for fim de string e as strings não são iguais ($CX \neq 0$ e $ZF=0$)

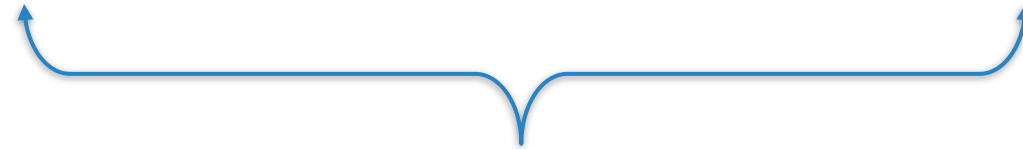
INSTRUÇÕES DE MANIPULAÇÃO DE STRINGS

Exemplo 1

```
MOV CX, 100  
CLD  
LACO: MOVSB  
LOOP LACO;
```

Exemplo 2

```
MOV CX, 100  
CLD  
REP MOVSB
```



Códigos equivalentes

INSTRUÇÕES DE MANIPULAÇÃO DE STRINGS

Programa que substitui a letra B pela letra C

```
DATA1 DB  'ABABABA', '$'
.CODE
        MOV AX, DS           ; Cópia DS para
        MOV ES, AX          ; ES
        CLD                  ; DF=0
        MOV DI, OFFSET DATA1
        MOV CX, 07           ; número de caracteres da string
        MOV AL, 42           ; caractere a ser comparado
LACO:   REPNE SCASB           ; começa analisar AL=? ES:[DI]
        JNE OVER            ; se z=0 (se B não for encontrado)
        DEC DI              ; z=1 (se encontrou B)
        MOV BYTE PTR [DI], 43
        JMP LACO
OVER:   MOV AH, 09
        MOV DX, OFFSET DATA1
        INT 21H              ; mostra a string resultante
```

INSTRUÇÕES DE MANIPULAÇÃO DE STRINGS

Rotina que recebe endereços de strings de 10 posições em DS:SI e ES:DI, compara os bytes e retorna em AL:

- 0 – se os bytes são iguais
- 1 – se os bytes são diferentes

```
INICIO:  PUSH CX
          MOV CX, 0AH    ; número de caracteres da string
          XOR AL, AL
          CLD             ; DF=0
          REPE CMPSB      ; repetir até que sejam diferentes (ZF=0) ou CX=0
          JZ  FIM
          MOV AL, 1
FIM:     POP CX
          RET
```

INSTRUÇÕES DE CONTROLE DO PROCESSADOR

Operações com Flags

CLI **STI**

CLD **STD**

CLC **STC**

CMC

Controle

NOP **HLT**

WAIT **LOCK** **ESC**

INSTRUÇÕES DE CONTROLE DO PROCESSADOR

Mnemônico	Significado	Formato	Operação	Flags afetados
CLD	Reseta DF	CLD	DF = 0	DF
STD	Seta DF	STD	DF = 1	DF
CLC	Reseta CF	CLC	CF = 0	CF
STC	Seta CF	STC	CF = 1	CF
CMC	Complementa CF	CMC	CF = CF'	CF
CLI	Reseta IF	CLI	Desabilita Interrupções IF = 0	IF
STI	Seta IF	STI	Habilita Interrupções IF = 1	IF

INSTRUÇÕES DE CONTROLE DO PROCESSADOR

Mnemônico	Significado	Formato	Operação	Flags afetados
NOP	Faz Nada	NOP	O MP faz 3 períodos de CLK inativo T_i	Nenhum
HLT	Halt	HLT	Interrompe processador até que a linha reset seja ativada, ou que uma interrupção não mascarável (ou mascarável, se permitidas) seja requerida	Nenhum
LOCK	Bloqueia Barramento	LOCK	Usada em aplicações de compartilhamento de recursos para assegurar que a memória não é acessada, simultaneamente, por mais de um processador. O barramento fica bloqueado até que a instrução seguinte tenha sido executada	Nenhum

INSTRUÇÕES DE CONTROLE DO PROCESSADOR

Mnemônico	Significado	Formato	Operação	Flags afetados
WAIT	Espera pela linha de teste ativa	WAIT	Faz o processador esperar até que um sinal no pino TEST' (verifica a cada $5T_{CLK}$). Se TEST' = 1, repete novamente, senão passa para o próximo comando	Nenhum
ESC	Escape	ESC	Solicita a um co-processador externo que execute uma instrução: COP: 11011 XXX O XXX representa a operação do co-processador	Nenhum